

setting enables all the stages that follow, and helps to access them as any other environment variable.

In contrast to the scripted syntax, each individual stage is grouped within a global and single enclosing block that is called 'stages', in the declarative syntax.

Pipeline script from SCM

Until the default checkout operation is disabled via `skipDefaultCheckout` (that should go in the *options* section), the pipeline will automatically clone the contents of the GitHub repository, as set up in the project configuration in Jenkins.

As shown in Figure 3, a project that was set up in the Jenkins instance managed by the author is configured to source the pipeline script from SCM—in this case, from a public Git repository hosted in GitHub. Jenkinsfile's name and path is set up in the *Script path* field, while the name of the branch name provided in the *Branches to build* helps to locate the script in that branch.

The *Additional behaviours* section (located just below the *Repository browser* field) helps to speed up the Git clone operation by setting options like minimising the *refspecs* to be used, timeout the operation, turn on or off the tags to download, etc. This is meant for power users and is a big time-saver if the repository to be cloned has a substantial history of *refspecs* (branches, tags, etc).

Git commit's SHA1

The first stage, *prep* (short for preparatory), runs a simple Git command, the output of which is processed further with

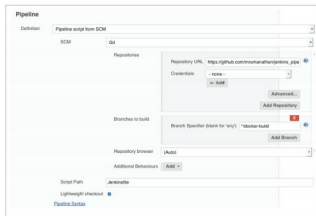


Figure 3: Pipeline configuration in Jenkins Project

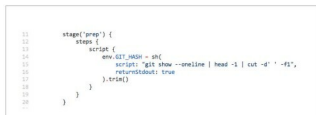


Figure 4: Retrieve the SHA-1 ID

the aid of basic UNIX commands to get the trimmed seven digit SHA1 of the Git commit. The output is stored in an environment variable as shown in Figure 4. While the Git command is comprehensible, there are a few other items in this code snippet that should be explained.

The entire Git command is contained within the *sh* step, and we will record the output (`returnStdout: true`) that we get after executing the command. The output from the command is passed through the *trim* function to a strip of extra space. This (*trim*) functionality is courtesy of the JIRA ticket of the Jenkins project.

Note: Any operation that demands usage of the Groovy language constructs within the pipeline script (applicable to only declarative syntax) should be enclosed in the script block. But how does one know when to or when not to use this block; flow control structures, variable definitions, function calls – all of these are fit to go inside the script block. And, of course, it is only by continuous practice that one gets to comprehend the usage of this block.

Access to Docker-related functions

With the installation of plugins (as listed in the *Plugins* section), the pipeline script has access to a host of Docker-specific build commands via the Docker variable (Figure 5 shows a list of variables offered by the Docker variable). Navigate to this page by clicking on the *Pipeline Syntax* option in the project page and selecting the option *Global Variables Reference* (shown in Figures 6 and 7).

Access to Docker commands, via the Docker variable, makes it very convenient to write simple pipeline script using the different functions, like `docker.build()`. Otherwise, this would have to be done by wrapping the regular Docker commands inside the *sh* step. For extensive



Figure 5: Functions accessible via the Docker variable



Figure 6: How to navigate the global variables section 1



Figure 7: How to navigate the global variables section 2